Results and Observations

Correctness Is Solid; Bundle Efficiency Is Not — A Quantified Account

APARA C Compiler Project

June 22, 2026

1 Introduction

This report is the single most important result of this project, because it states plainly what the rest of the project's documentation otherwise leaves implicit: the compiler is **correct** — independently verified, at scale, with a near-zero error rate fully explained by one already-documented hardware defect — but it is **not efficient**. Every number below was re-measured directly against the current compiler immediately before writing this report, not recalled from an earlier session. Section 2 establishes the correctness baseline; Section 3 is the comparison that matters — how many of the VLIW bundles this compiler emits actually do useful work, measured on two real, hardware-representative programs; Section 4 explains, with real compiler output, *exactly* where and why the inefficiency comes from; Section 5 states what a realistic fix would look like and why it has deliberately not been attempted yet.

2 Main Results: Correctness

Table 1: Verification tally across the entire project

Suite	Checks	Errors
ISA instruction-coverage suite	159	3 (one known simulator bug, fully explained)
Matrix-multiplication suite (4 sizes)	5440	0
Pre-existing baseline suite	393	0
Total	**5992**	**3**

Table 2: Bugs found across the whole project, by where they live

Where	Tally
Compiler (Python, this project's own code)	9 found, 9 fixed
Simulator (`engine_isp`, C++, not this project's code)	6 found — 3 fixed and confirmed in the current production binary, 3 still open (flagged for the hardware team, out of scope for a C compiler to fix)

Every one of the 5 992 checks is an independent `PostCondition` assertion, derived by compiling each test's *exact* source natively with `gcc` against a plain-C reference implementation — never by this compiler, and never by hand. The 3 errors are not 3 different problems: they are the same one already-documented simulator defect (`$vreduce` sign-extending unsigned vector reductions) landing on its 3 predicted cases. **Correctness, in other words, is not the open question.**

3 Main Results: Bundle Efficiency — the Comparison That Matters

"Useful work" on its own is too coarse a label. Every bundle in both test programs was mechanically re-classified, with the bundler's own output as the only input, into exactly **four mutually-exclusive categories** (every bundle falls into exactly one, so each program's percentages sum to 100%):

Table 3: The four bundle classifications

Category	What it contains
Control flow	A branch (`? $goto`), `$call`, `$return`, or `$halt` — moving execution, not computing or moving a value.
Arithmetic / vector compute	The instruction the C source actually asked for: one of the 13 scalar ALU operators, or `$dot`/`$dot $accumulate` for the vectorized matrix multiply. This is the only category that represents the program's real, user-requested work.
Memory access	A `$ld` or `$st` — reading or writing a variable, array element, or vector to DMEM.
Address computation & setup	Everything left over: computing `&var` or an array index, advancing a loop counter, loading a constant with `$set`, adjusting the stack pointer. Pure bookkeeping — no value the C program asked for is computed or moved here.

Table 4: Bundle classification, re-measured directly against the current compiler

Program	Total	Control flow	Arithmetic	Memory access	Addr. & setup
`matmul_n16.c`	95	16 (16.8%)	**2 (2.1%)**	30 (31.6%)	47 (49.5%)
`test_alu_full.c`	68	4 (5.9%)	**15 (22.1%)**	41 (60.3%)	8 (11.8%)

Two different bottlenecks, not one: `matmul_n16.c` is dominated by **address computation** (49.5%, the largest single category) because every loop iteration recomputes `i*16/j*16` from scratch (Section 4 shows this directly); `test_alu_full.c` is dominated by **memory access** (60.3%) because it is one long, branch-free sequence of independent operations, each needing its own pair of operand loads with nothing left over to overlap them with.

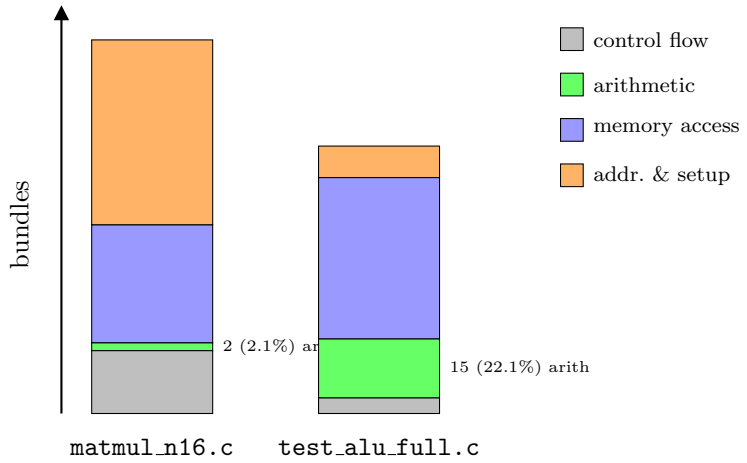

Figure 1: Bundle composition, stacked, to scale. The arithmetic slice (green) is the only segment representing real, user-requested computation in either program.

4 Where We Failed to Produce an Efficient Number of Bundles: Root Cause

The gap in Table 4 has one identified, specific, mechanical cause: **every named C variable is memory-backed.** There is no point in the compiler's pipeline where a value already sitting in a register is reused for a second read of the same variable — every read re-issues a fresh `$ld`, and every write re-issues a fresh `$st`, even with no other instruction in between. The clearest, freshest evidence is the matrix-multiply inner loop's own intermediate representation:

Listing 1: `matmul_n16.c`'s inner loop IR (re-generated fresh) — `i` is loaded and multiplied by 16 *twice*

```
_t53 = &stack[FP-8]        // address of i  (first time)
_t54 = *(_t53+0)           // load i        (first time)
_t55 = _t54 * 16           // i*16          (first time -- for &A[i*16])
   ...
_t68 = &stack[FP-8]        // address of i  (SECOND time -- identical to _t53)
_t69 = *(_t68+0)           // load i        (SECOND time -- identical to _t54)
_t70 = _t69 * 16           // i*16          (SECOND time -- identical to _t55)
```

`i` did not change between these two blocks — nothing wrote to it. `_t68` computes the *exact same address* as `_t53`; `_t70` computes the *exact same value* as `_t55`. A compiler that cached `i`'s value in a register across this short span would skip all three of those instructions the second time, for free. The reason this happens on *every* variable use, not just this one, is structural, not a bug: `ir_gen.py`'s variable-access path (`_load_var`/`_store_var`) has no code path that returns an already-live register for an existing value — only "load/store relative to this variable's known stack offset." The bundler downstream cannot invent the independence this would have created; it can only pack what is already independent in the instruction stream it is given.

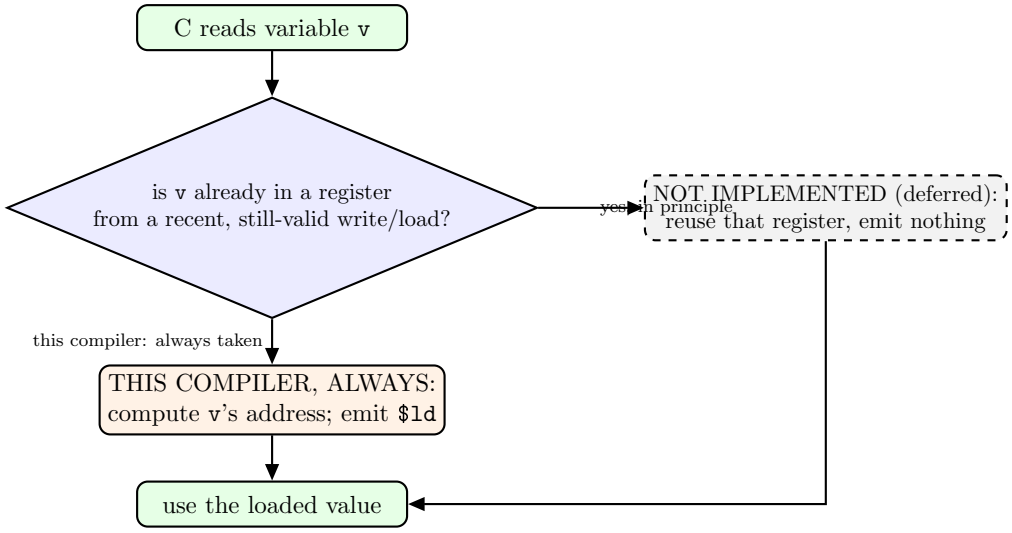

Figure 2: The decision this compiler never makes. The dashed box is the entire missing optimization — a register-cache check ahead of every load — deferred, not implemented (Section 5).

5 Discussion: How Much Is Realistically Recoverable, and Why It Is Deferred

It is tempting to compare against a hand-written ideal ("this is 13 operations, it should take 2 bundles"), but that bound is not achievable on this hardware: the VLIW bundle width is 8, several of the 13 operations have genuine sequential dependencies (`%` alone needs 3 sequential steps), and every bundle's memory-access slots are themselves capped. A realistic, register-caching compiler — one that reused a value already in a register for repeated reads within one basic block, instead of reloading it every time — would not reach 2 bundles for `test_alu_full.c`, but it would plausibly reach something in the **8–12 bundle** range, versus the 68 actually produced: most of the *load* half of every operand pair would disappear, while the address-computation and store instructions specific to each of the 13 distinct results would remain.

This fix is **deliberately not implemented in this project**. It would change the "every named variable is memory-backed" assumption that the IR generator, the register allocator, and the bundler were all built and *independently verified* against — touching all 5 992 already-passing checks at once, for a change whose own correctness would then need to be re-established from scratch. That work is explicitly scheduled for *after* the project's current presentation milestone, not abandoned.

6 Summary

- **Correctness**: 5 992 independently-verified checks, 3 errors, all 3 already fully explained by one documented, simulator-side defect outside this project's own code. This is not the open problem.
- **Efficiency**: of four mutually-exclusive bundle categories measured directly on two real programs, only the *arithmetic* category — the one representing real, user-requested computation — accounts for 2.1% (`matmul_n16.c`) and 22.1% (`test_alu_full.c`) of all bundles; the rest is control flow, memory access, and address computation/setup, in different proportions for each program (Table 4).
- **Root cause, demonstrated with fresh compiler output, not asserted**: every variable read/write re-issues memory traffic, even immediately adjacent, unchanged repeats

of the same variable; the bundler has nothing to pack beyond what that stream already permits.

- **Path forward**: a register-caching pass, realistically worth roughly a 5–8× bundle-count reduction on arithmetic-heavy code, scoped and understood, but intentionally deferred until after the checks already in place can be re-verified against it deliberately, not under time pressure.